



NuGraph NEXUS

Shahram Ghandeharizadeh

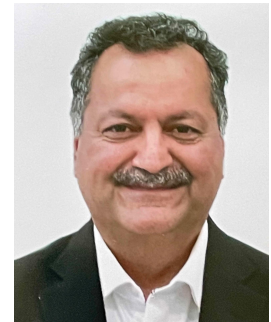
Team Members



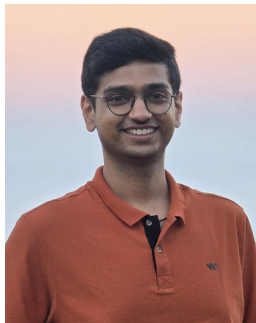
Hieu



Jun



Shahram



Lakshya



Shi Hao



Minho



Xinjie

Outline

- Motivation: Performance
 - The Need for Speed!
- NEXUS: Overall Architecture
- Parallelism:
 - Pipelining
 - Intra-query
 - Inter-query
 - All of the above
- Shortest-path & declarative pattern matching
- Conclusions and Future Research

Motivation: eBay Graph Queries

Read-only/intensive workloads consisting of simple and complex operations.



eBay Graph Queries: The Need for Speed!

Read-only/intensive workloads consisting of simple and complex operations.



Complex eBay Queries

```
g.V().has("User","id",?).
  local(__.outE("useSignInIpAttempt").order().by("
    lastModifiedTime",Order.desc).limit(64L).has
    ("lastModifiedTime",?)).inV().dedup().
  local(__.inE("useSignInIpAttempt").order().by("
    lastModifiedTime",Order.desc).limit(64L).has
    ("lastModifiedTime",?)).outV().elementMap("
    lastSuspensionTime").dedup().
  where(__.select("lastSuspensionTime").is(?)).
    count()

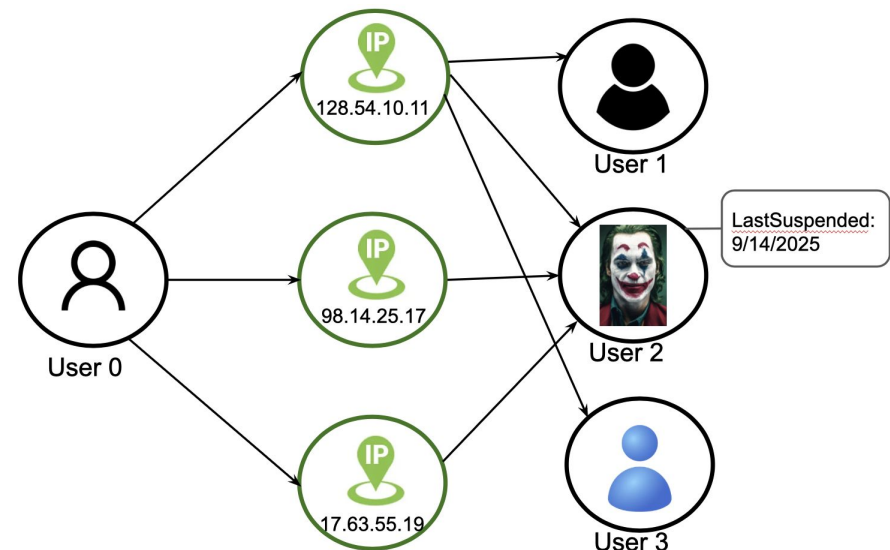
g.union(
  __.V().has(\"graph_id\",?).has(\"entity_id\",?).
    limit(1L)).coalesce(__.local(__.outE(?).has
    (\"probability\",P.gt(?)).order().by(\"
    probability\",Order.desc)).project(\"
    destinationProps\", \"edgeProps\", \"
    sourceProps\").by(__.inV().valueMap(\"
    entity_id\", \"count\", \"prefLabel\", \"type
    \", \"image_uri\")).by(__.valueMap(\"type
    \", \"probability\")).by(__.outV().valueMap
    (\"entity_id\", \"count\", \"prefLabel\", \"
    type\", \"image_uri\")).__project(\"
    sourceProps\").by(__.valueMap(\"entity_id
    \", \"count\", \"prefLabel\", \"type\", \"
    image_uri\"))
)
```

```
g.V().match(
  __.as("VendorAPI").has("vtype","ebayontl:
    VendorAPI"),
  __.as("VendorAPI").out("ebayontl:
    belongToInfohubDomainRelatedToVendorAPI").as
    ("InfohubDomain"),
  __.as("VendorAPI").out("ebayontl:
    hasContractInformationRelatedToVendorAPI").
    as("ContractInformation"),
  __.as("ContractInformation").values("ebayontl:
    ContractInformationId").as("
    ContractInformationId"),
  __.as("InfohubDomain").values("ebayontl:
    InfohubDomainId").as("InfohubDomainId")
).select("ContractInformationId","InfohubDomainId")
```

Fraud Detection: 95th Latency Faster than 50 milliseconds

- A query from eBay Risk application: Given a user, say User 0, check whether the user is suspicious via IP address linking.

```
g.V().has("User", "id", ?) .
  local(__.outE("useSigninIpAttempt").order().by("
    lastModifiedTime", Order.desc).limit(64L).has
    ("lastModifiedTime", ?)).inV().dedup().
  local(__.inE("useSigninIpAttempt").order().by("
    lastModifiedTime", Order.desc).limit(64L).has
    ("lastModifiedTime", ?)).outV().elementMap("
    lastSuspensionTime").dedup().
  where(__.select("lastSuspensionTime").is(?)).
  count()
```



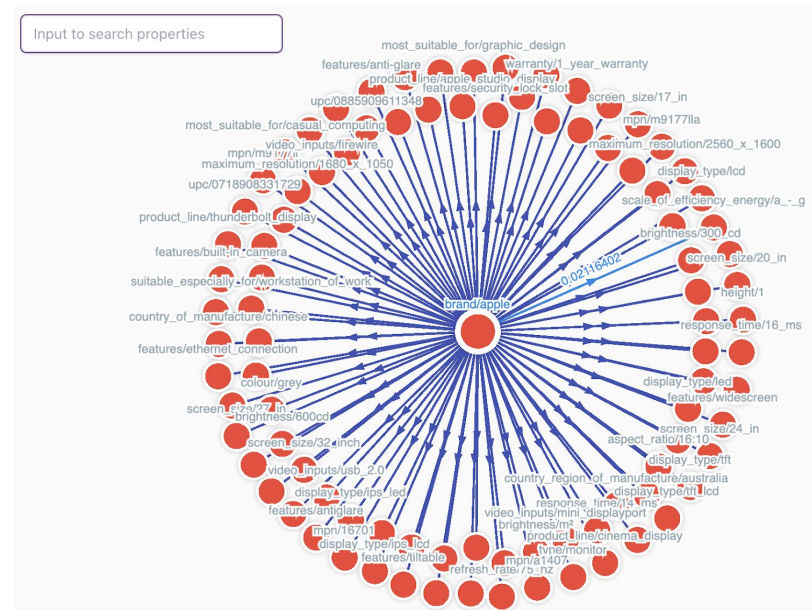
The same query may use other properties: Address, Email, Device, Payment, Phone, GovIssued, etc.

Search/Recommendation: 99th latency faster than 50 msec

When a user views a specific item, such as an iPhone, the query generates a list of alternative models (such as the Google Pixel or Samsung Galaxy) as personalized recommendations.

```
g.union(  
  __.V().has(\"graph_id\",?).has(\"entity_id\",?).  
    limit(1L)).coalesce(__.local(__.outE()).has  
    (\"probability\",P.gt(?)).order().by(\"  
    probability\",Order.desc)).project(\"  
    destinationProps\", \"edgeProps\", \"  
    sourceProps\").by(__.inV().valueMap(\"  
    entity_id\", \"count\", \"prefLabel\", \"type  
    \", \"image_uri\")).by(__.valueMap(\"type  
    \", \"probability\")).by(__.outV().valueMap  
    (\"entity_id\", \"count\", \"prefLabel\", \"  
    type\", \"image_uri\")).__project(\"  
    sourceProps\").by(__.valueMap(\"entity_id  
    \", \"count\", \"prefLabel\", \"type\", \"  
    image_uri\"))  
)
```

Hundreds of millions of vertices.



- Middle vertex: Found via (graph_id, entity_id), e.g., iPhone.
- Other vertices are connected to the middle vertex via edges; "weight" on edges expressed in the "probability" property.

NEXUS

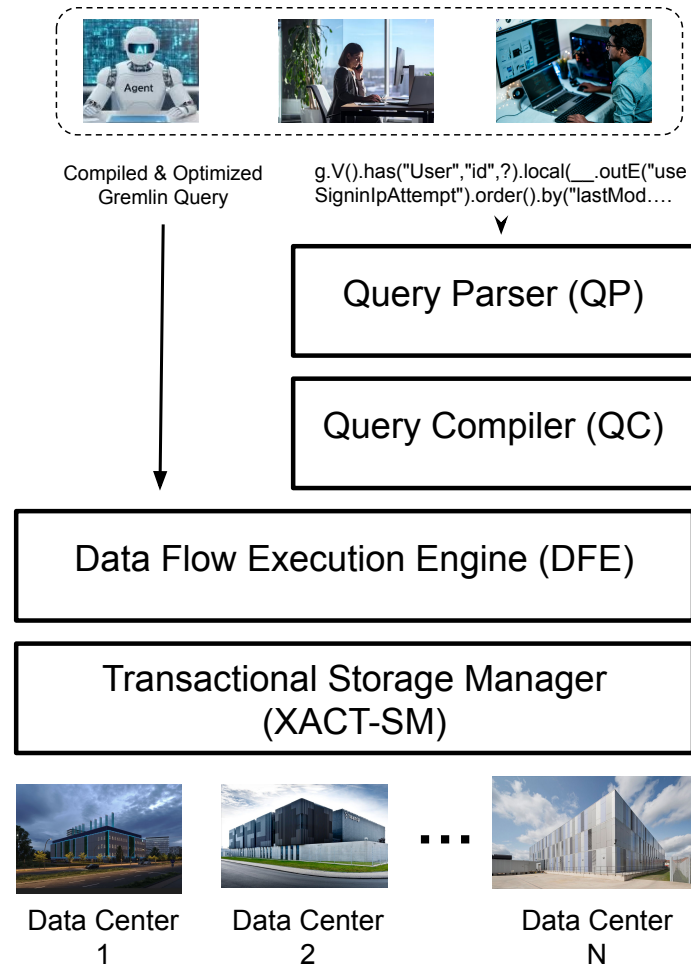
- NEXUS is a high performance query processor for the Gremlin query language.
 - Objective: Fast response times for both simple and complex queries.
- Targets graph databases. Why?
 - Superior performance for connected data: Implicit joins.
 - Flexible schema and agility: Add new vertex types, edges, and properties dynamically.
 - Powerful for relationship-centric use cases: Social networks, fraud detection, recommendation engines, knowledge graphs, etc.
 - Contextual awareness and pattern discovery: Suitable for applying graph algorithms.

NEXUS

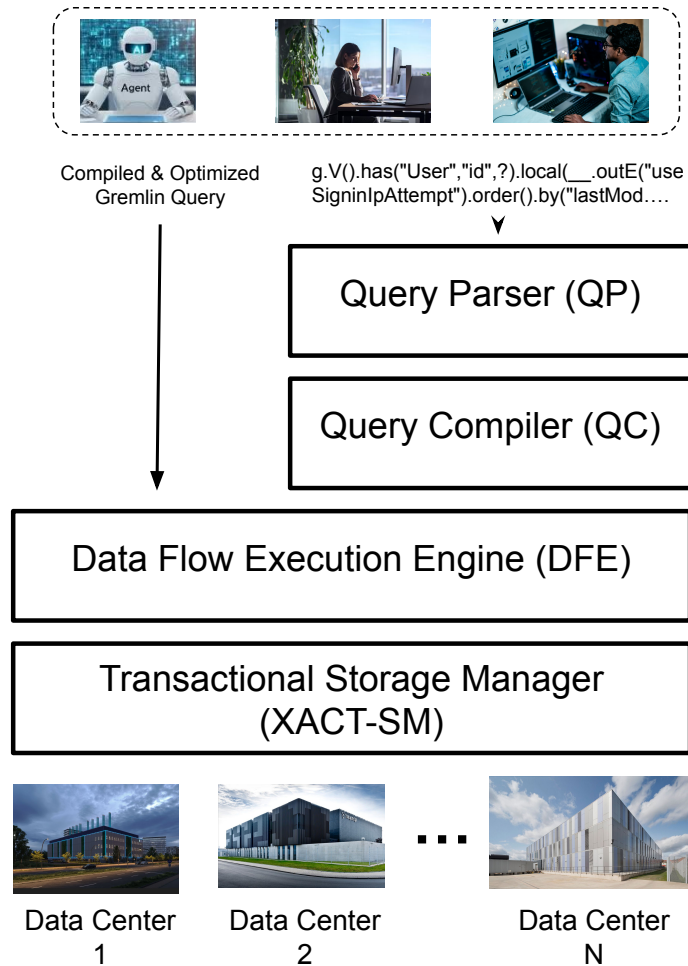
- NEXUS is a high performance query processor for the Gremlin query language.
 - Objective: Fast response times for both simple and complex queries.
- Targets graph databases. Why?
 - Superior performance for connected data. No joins.
 - Flexible schema and agility: Add new vertex types, edges, and properties dynamically.
 - Powerful for relationship-centric use cases: Social networks, fraud detection, recommendation engines, knowledge graphs, etc.
 - Contextual awareness and pattern discovery: Suitable for applying graph algorithms.
- What is new?
 - Parallelism.
 - A disaggregated dataflow architecture.
 - 1-hop query result caching.
 - eBay's production workload to evaluate our algorithms.



NEXUS: Architecture



NEXUS: Architecture (Cont...)



We presented DiStash XACT-SM at the TPCTC workshop on September 1, 2025 at London/VLDB.

NEXUS: Overview

Query Compiler:



Gremlin
Query



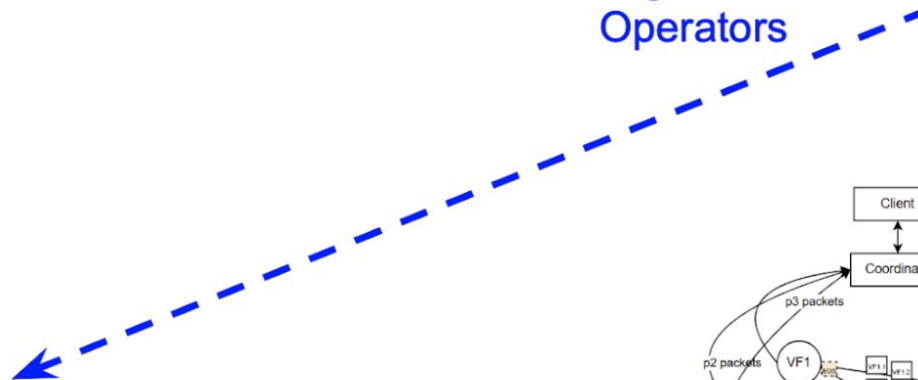
TinkerPop
Representation



NEXUS Query
DAG
+
Algebraic
Operators



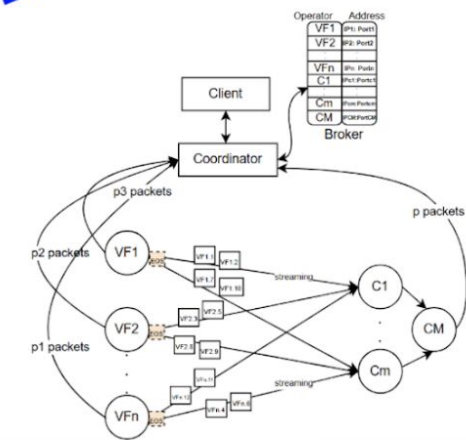
NEXUS DFE
Physical Plan



Query1.NEXUS



Parameterized
Execution



Data Flow Execution

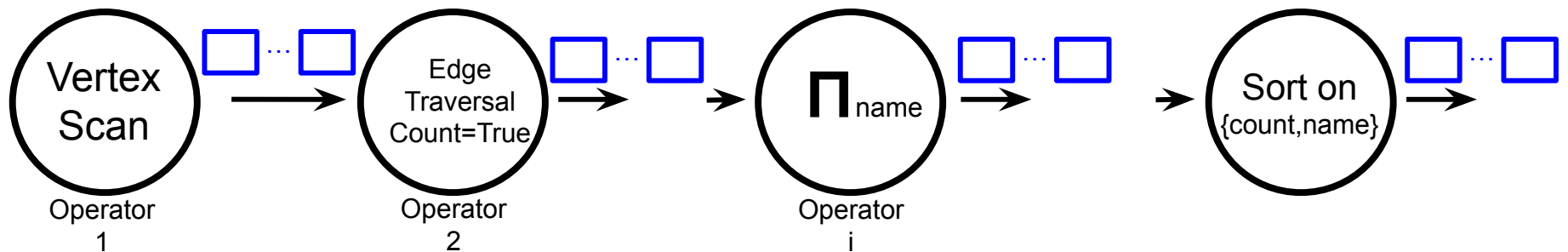
NEXUS: A 9 Month Old Research Prototype



- NEXUS is a prototype for testing a hypothesis.
 - Hypothesis: Feasibility of a data flow execution paradigm for processing both simple and complex Gremlin queries to improve service time and resource efficiency.
 - There was a lot of skepticism about such an implementation when we started in January of 2025. Why? eBay's Gremlin queries are intimidating even to the experts.
- NEXUS is designed to be thought provoking and to stimulate new system designs for processing graph queries.
- NEXUS demonstrates the viability of design concepts that justify an implementation.
- NEXUS compiler is implemented using Java. Its data flow execution engine is implemented using Vert.x, an event-driven framework using Java. We present preliminary performance numbers to highlight tradeoffs.

NEXUS: Concepts

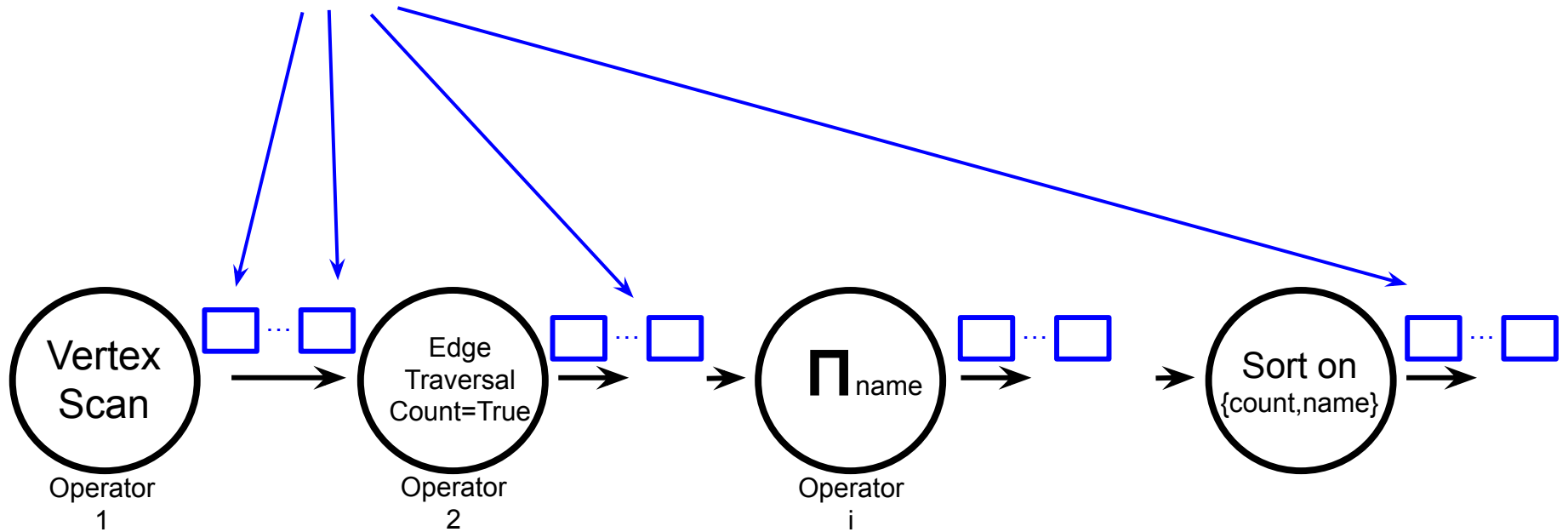
- Operator: A Vert.x Vertical. Implements a simple operation on a data element, e.g., scan vertices/edges, count number of edges of a vertex, lookup a property value of a vertex, etc.
- Pipelining: Overlapped processing. Given a query, its Operator i processes data as its Operator 1 produces data.



- Data flow: Data is flowing through the different operators in the form of network [packets](#).

NEXUS: Network Packets

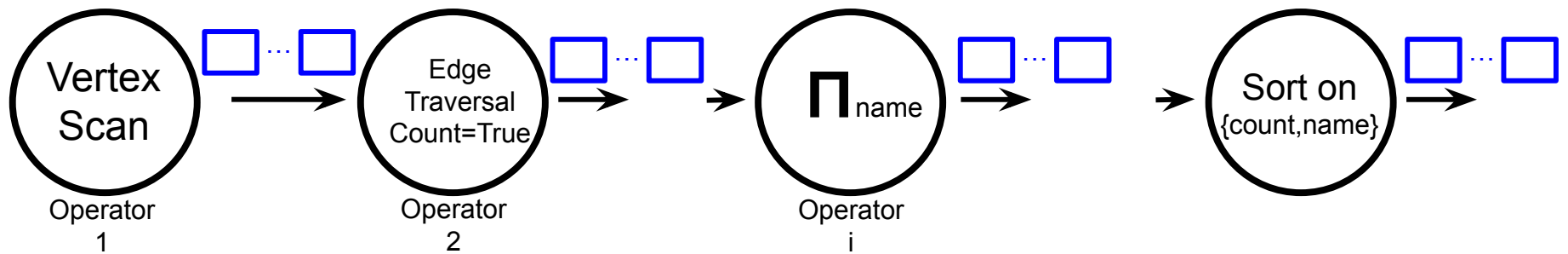
- A network **packet** consists of both metadata and data.



- Metadata: Query DAG + execution instructions.
- Data: Intermediate or final results produced by an operator.

NEXUS: Parallelism

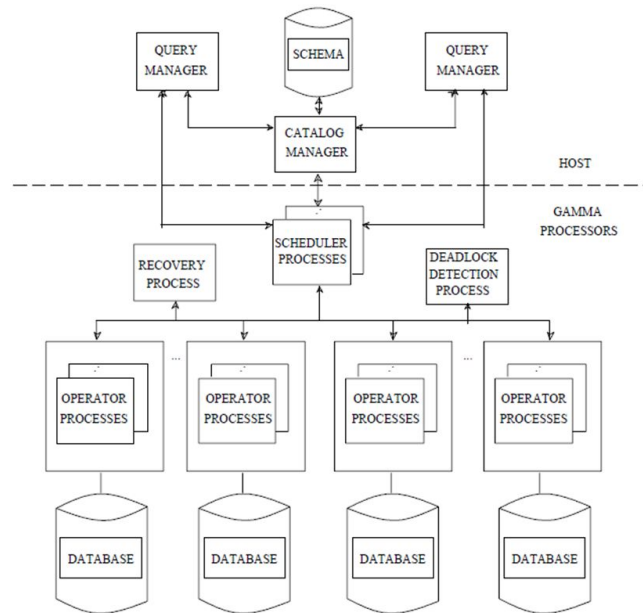
- Parallelism to enhance performance:
 - Pipelining,
 - Intra-operator parallelism,
 - Inter-query parallelism,
 - All of the above.



NEXUS Hides Parallelism Using Gremlin

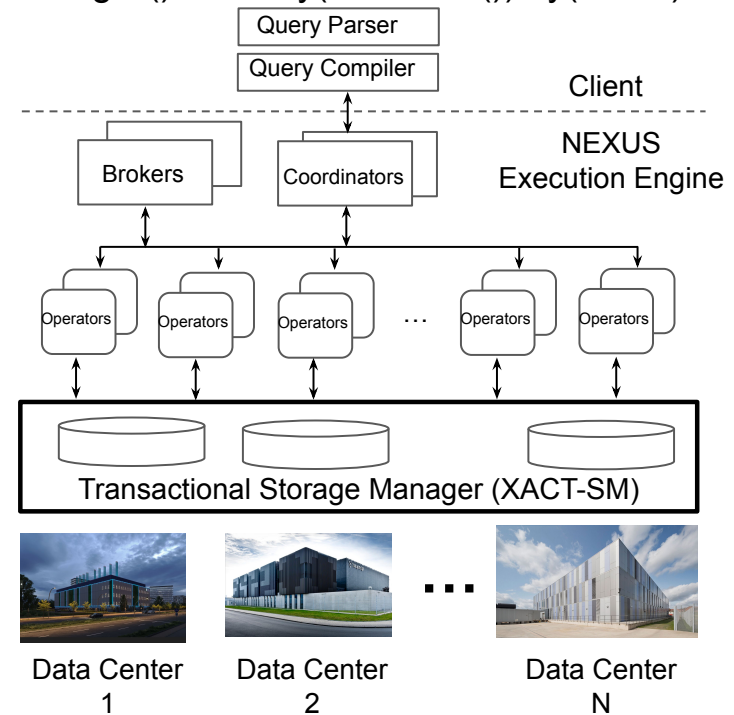
SQL

SELECT *
FROM Emp
WHERE salary=60K



Gamma, 1985-1990

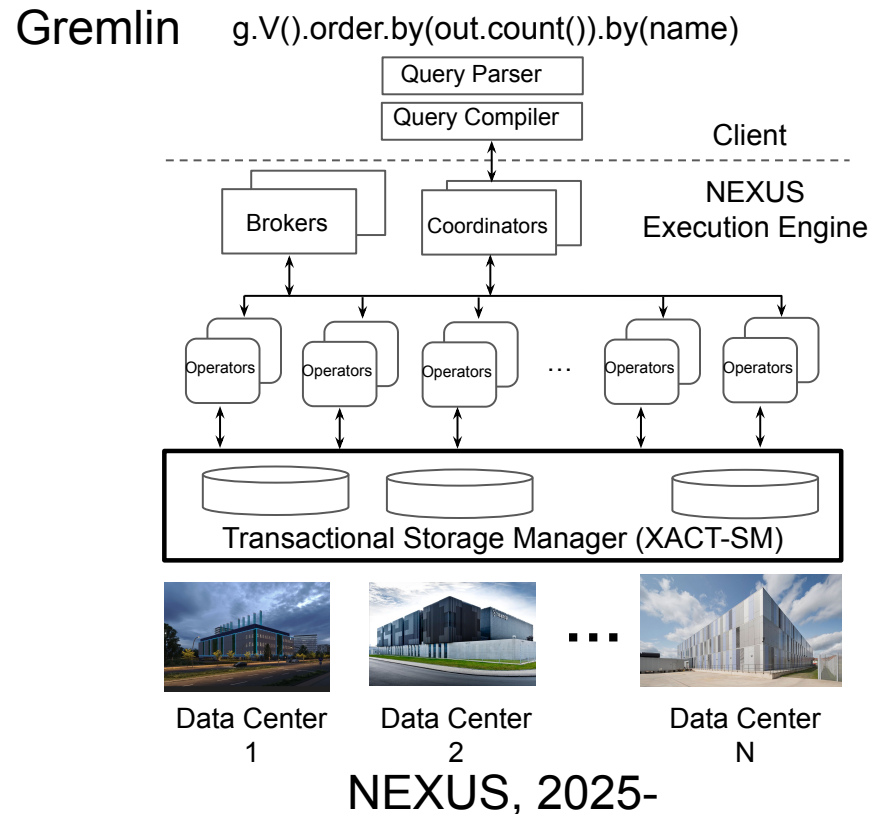
Gremlin: `g.V().order.by(out.count()).by(name)`



NEXUS, 2025-

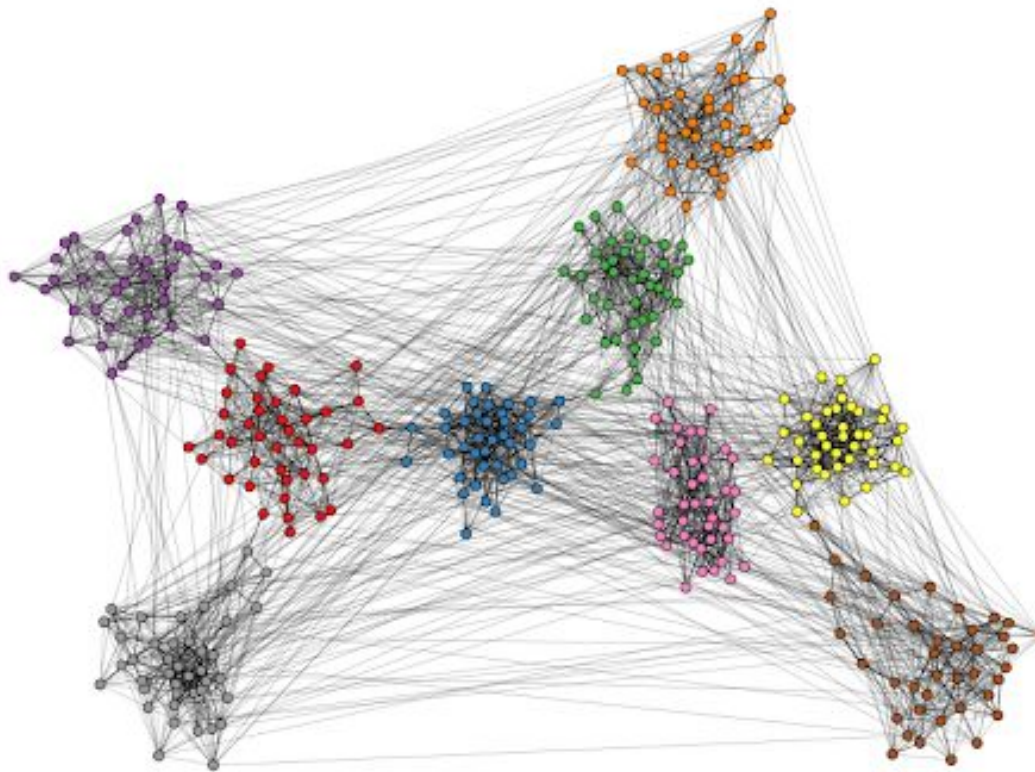
NEXUS Data Flow Framework

- An operators registers itself with all brokers.
- Brokers facilitates elasticity and discovery of operators. They may launch operators when the load is high.
- A coordinator schedules the operators of a query.
- A client issuing a simple query may bypass the query parser, compiler, and coordinator to access an operator directly.



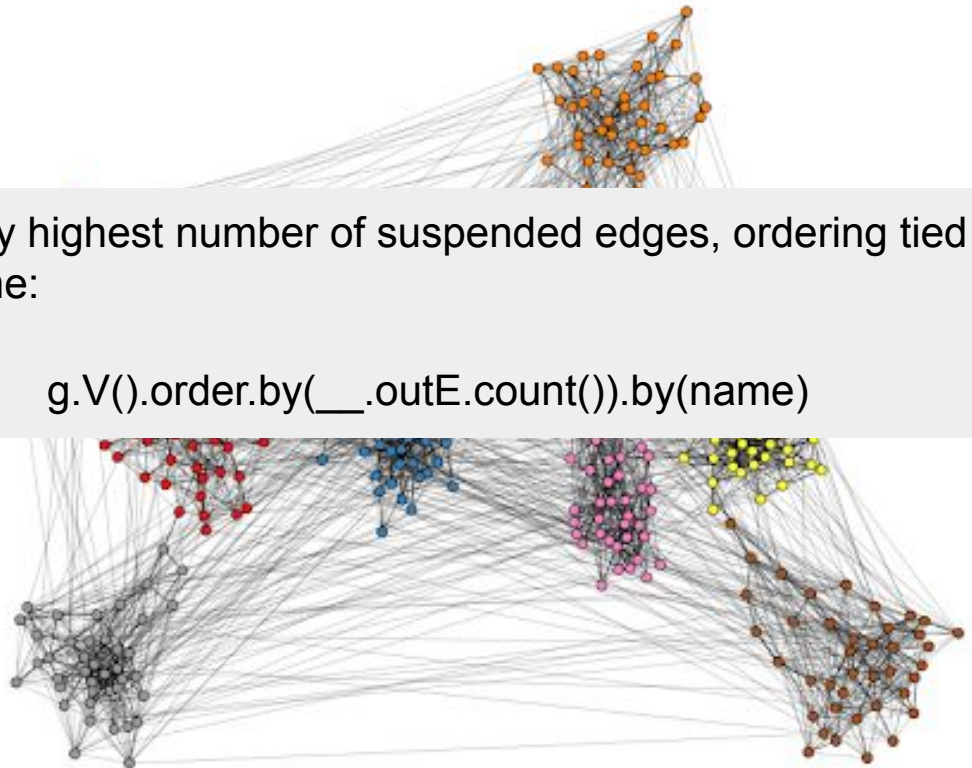
Example: Order Vertices Using Number of Outgoing Edges

Applications: Fraud detection, social networking, supply-chain, etc.



Example: Order Vertices Using Number of Outgoing Edges

Applications: Fraud detection, social networking, supply-chain, etc.

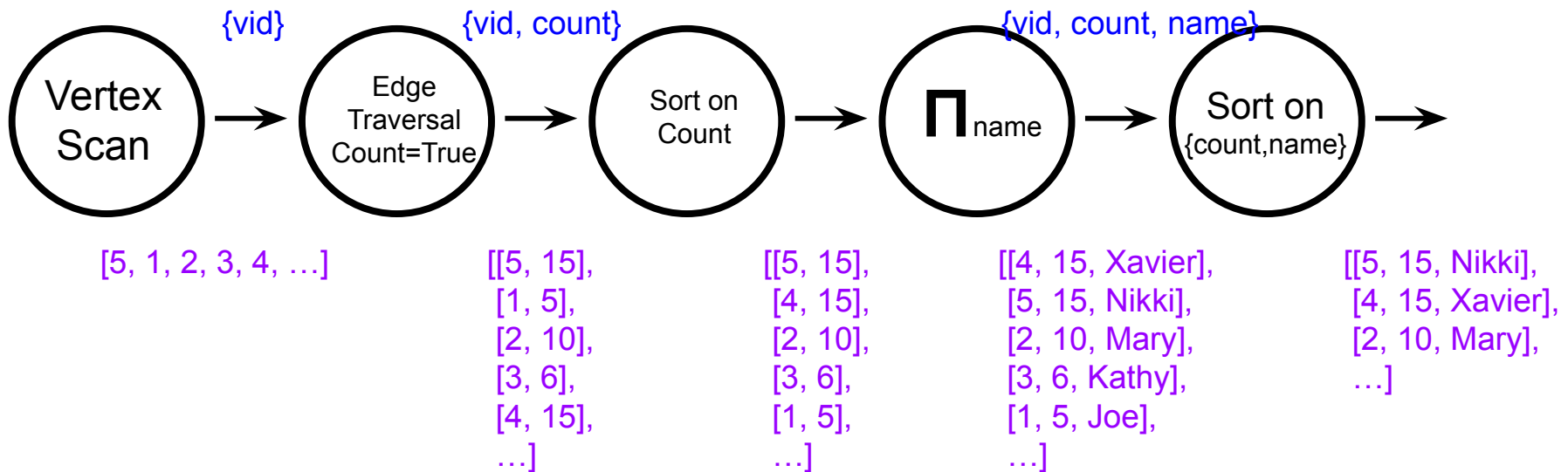


List eBay members by highest number of suspended edges, ordering tied members alphabetically by name:

```
g.V().order.by(__.outE.count()).by(name)
```

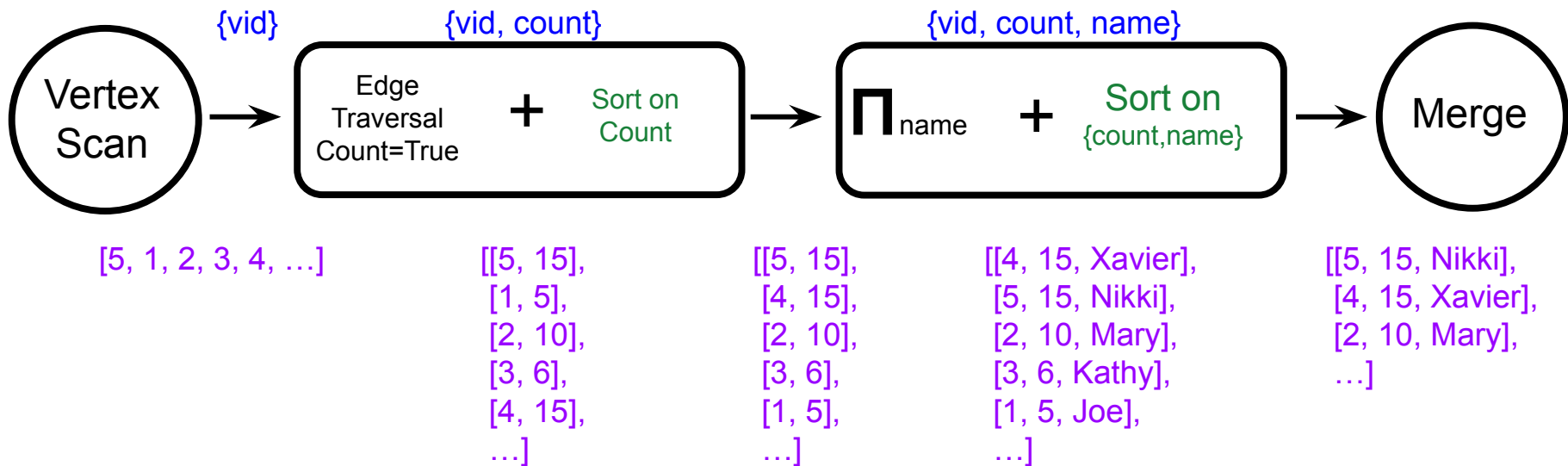
Complex Queries: Pipelined Sort Execution (Top-K)

```
g.V().order.by(__.outE().count).by(name)
```



Complex Queries: Pipelined Execution

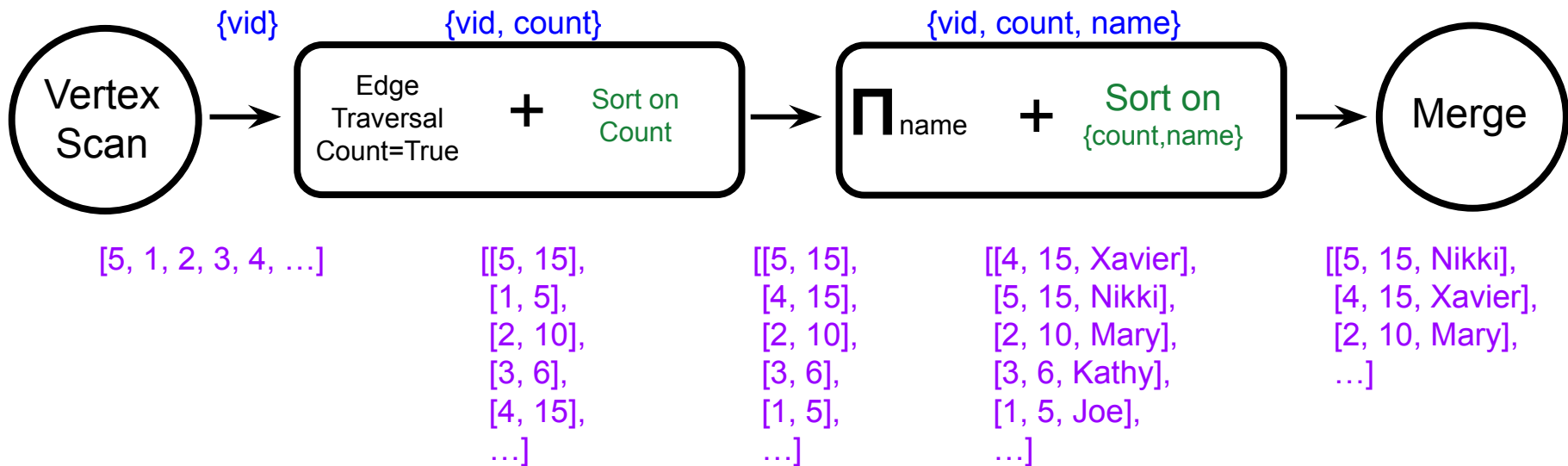
```
g.V().order.by(__.outE().count).by(name)
```



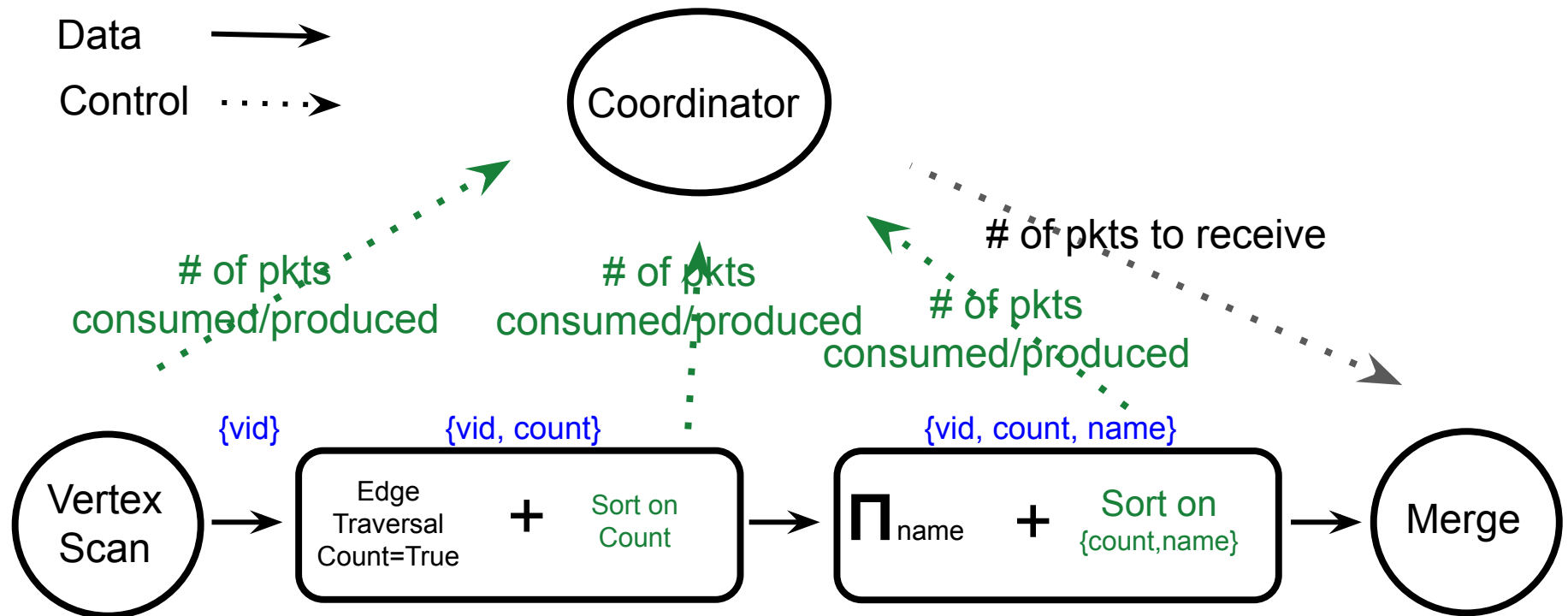
Complex Queries: Pipelined Execution

Merge is a barrier operator.

```
g.V().order.by(__.outE().count).by(name)
```



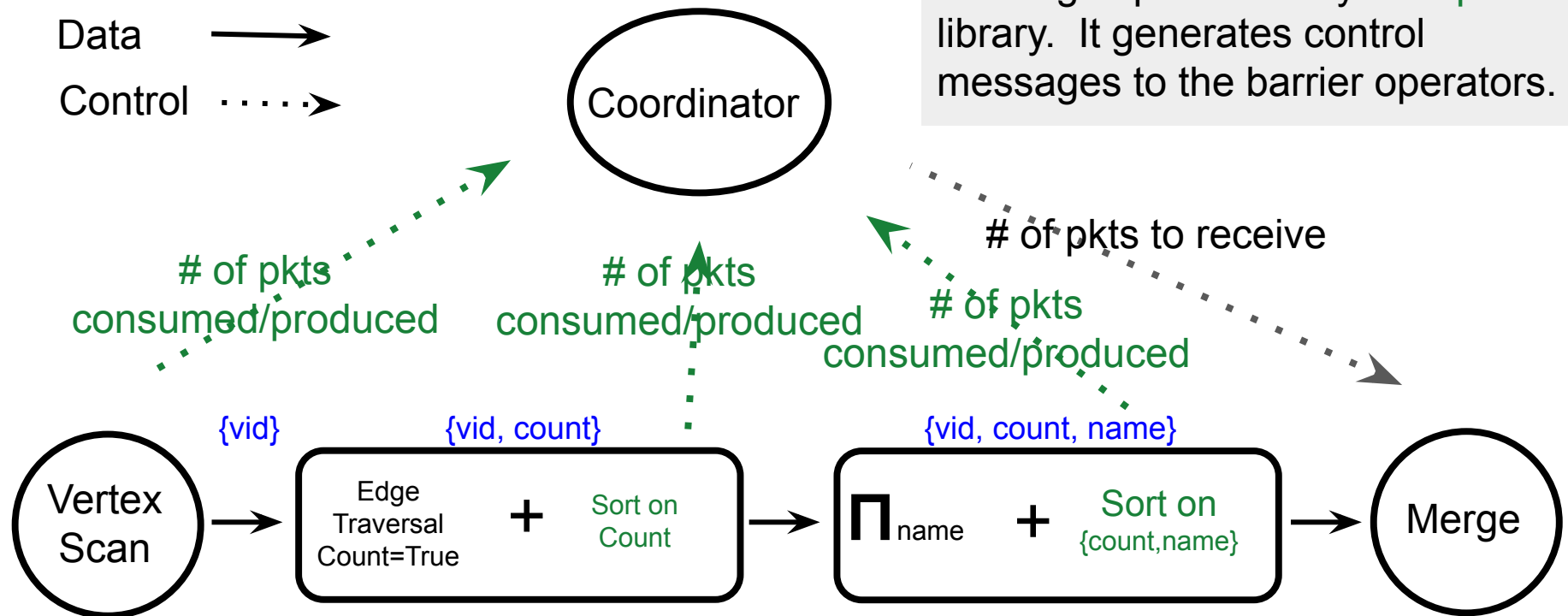
Barrier Operator: How Many Packets to Receive?



Barrier Operator

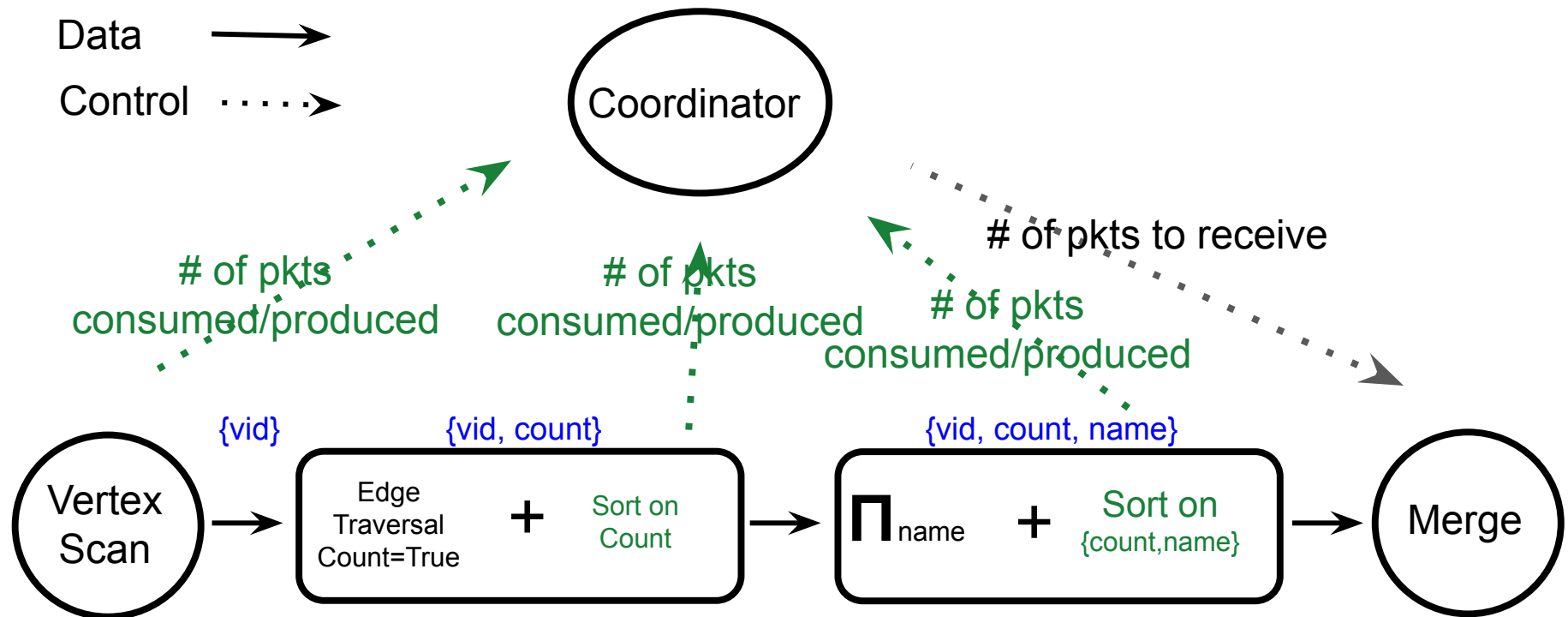
Data $\longrightarrow$
Control $\cdots\cdots\longrightarrow$

Data flows between operators. The coordinator processes control messages produced by the **split** library. It generates control messages to the barrier operators.



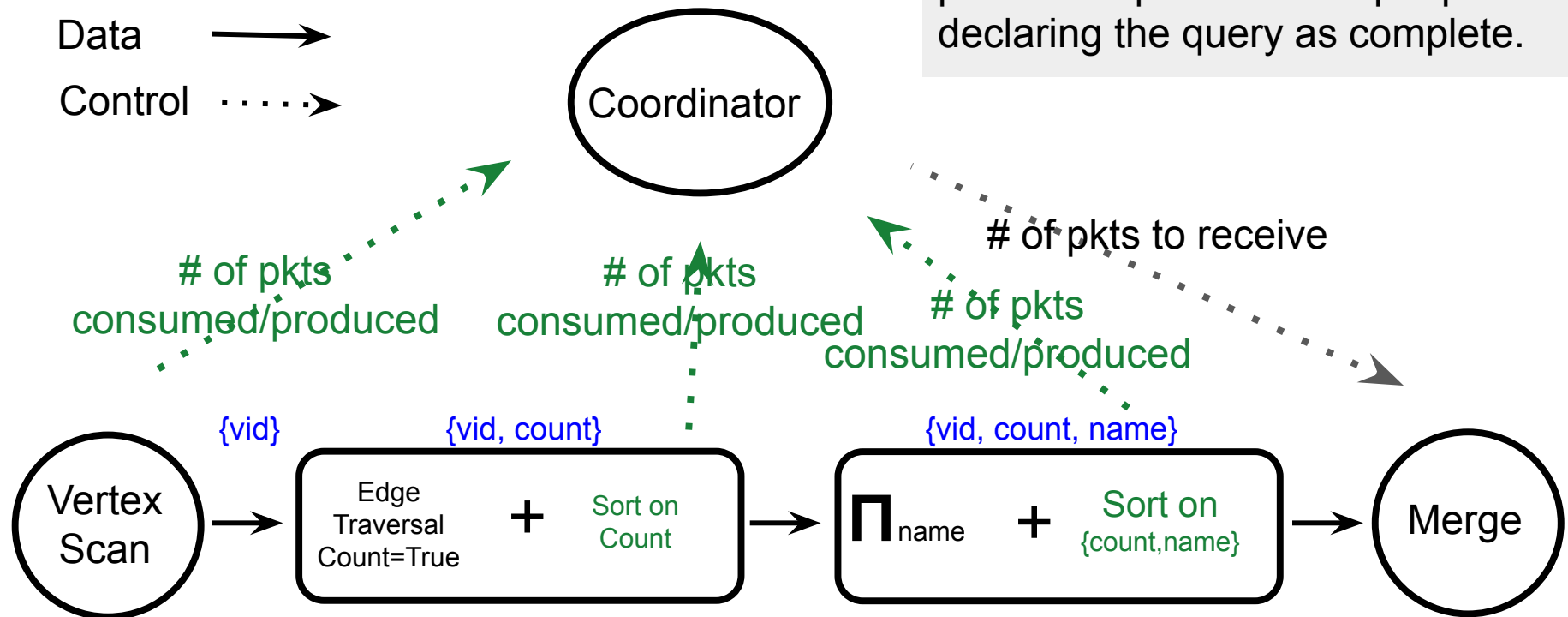
Barrier Operator

Many coordinators. One coordinator per query.

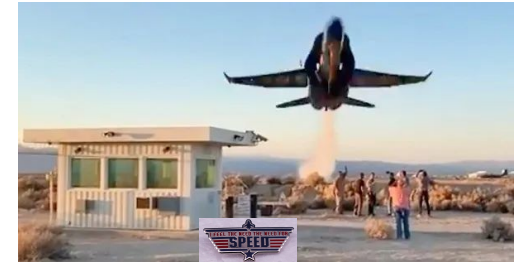


Query Termination

Client is a barrier operator.
Coordinator tells it how many packets to process as input prior to declaring the query as complete.



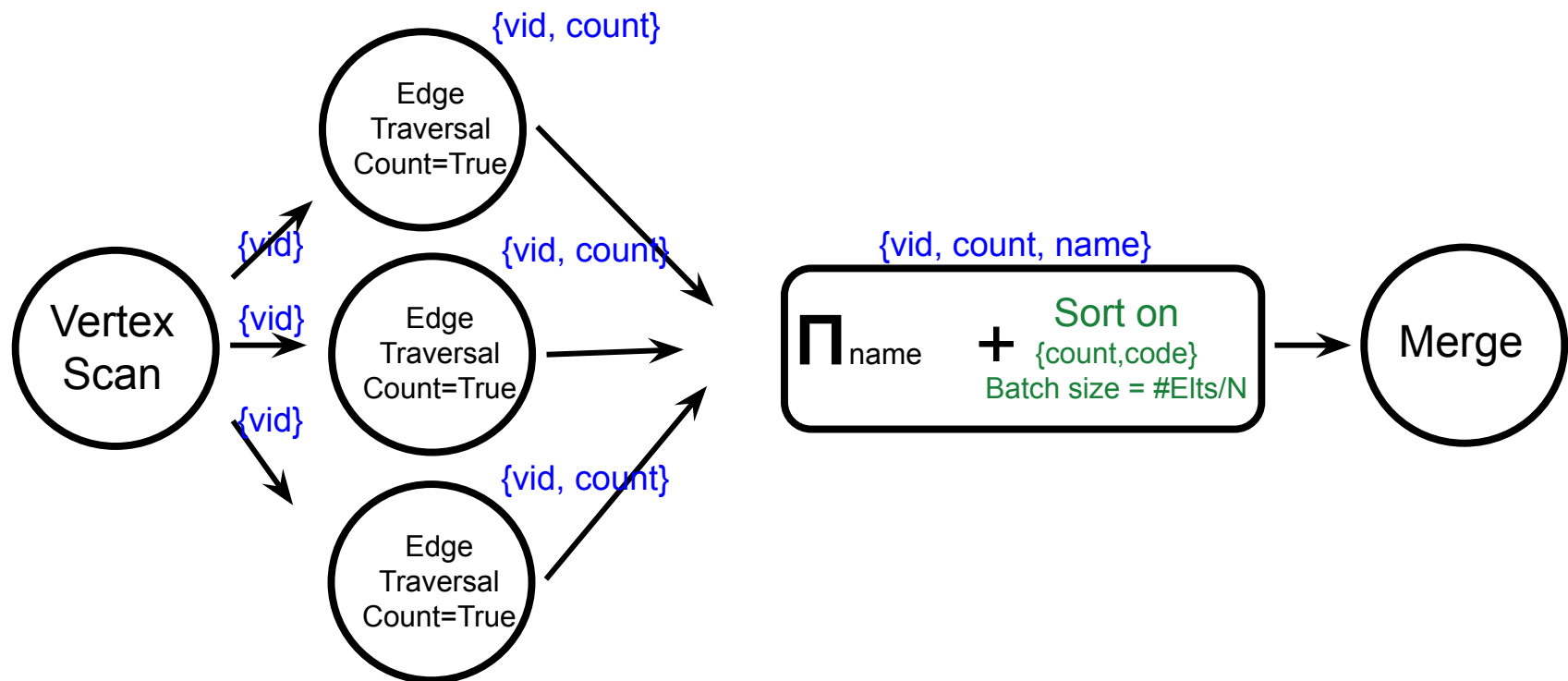
Parallelism in NEXUS



- **Intra-operator** parallelism:
 - Divide work of a query across different instances of the same operator.
- **Inter-operator** parallelism:
 - Different operators in the query process work concurrently.

Pipelined Execution + Intra-Operator Parallelism

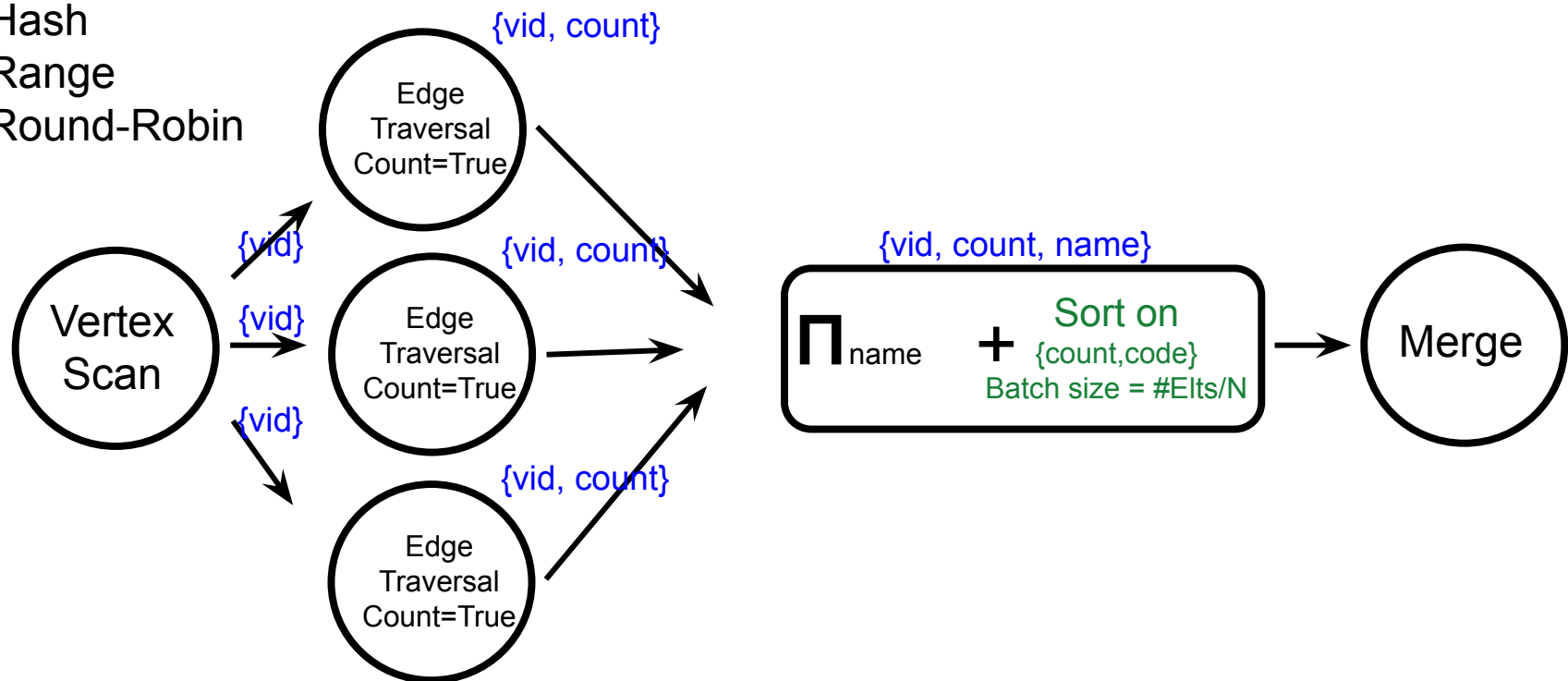
```
g.V().order.by(__.outE().count).by(name)
```



Intra-Operator Parallelism: How? Data partitioning.

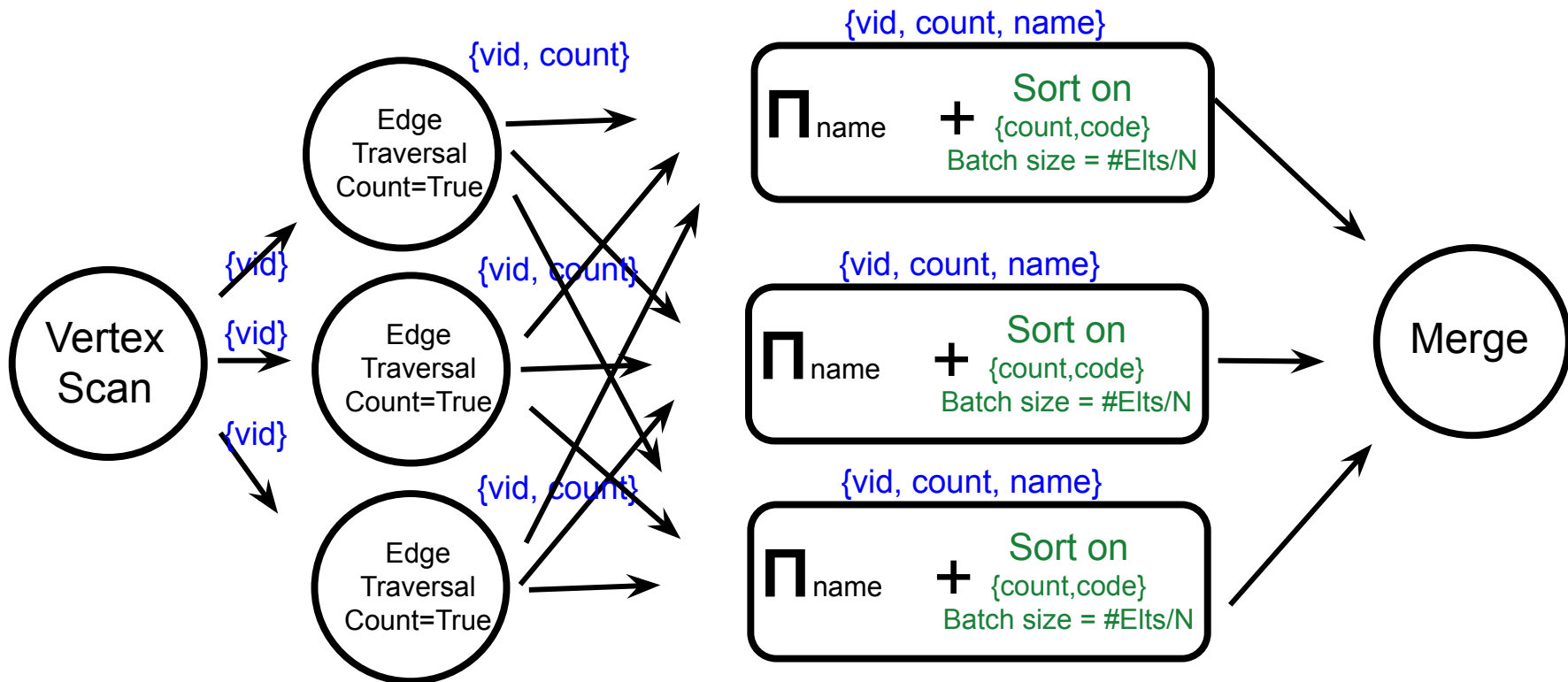
```
g.V().order.by(__.outE().count).by(name)
```

- Hash
- Range
- Round-Robin



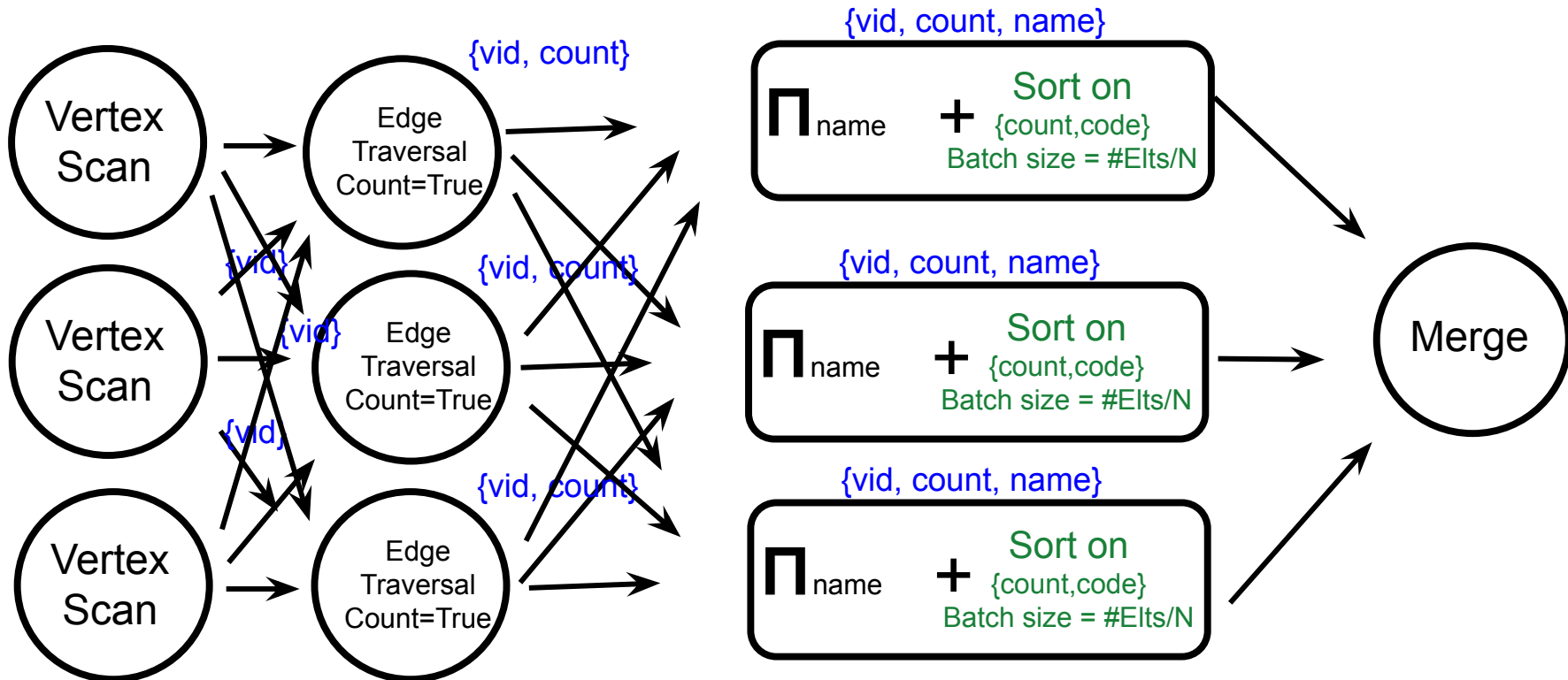
Intra-Operator Parallelism: Data Partitioning

```
g.V().order.by(__.outE().count).by(name)
```



Intra-Operator Parallelism: Concurrent Scans

```
g.V().order.by(__.outE().count).by(code)
```

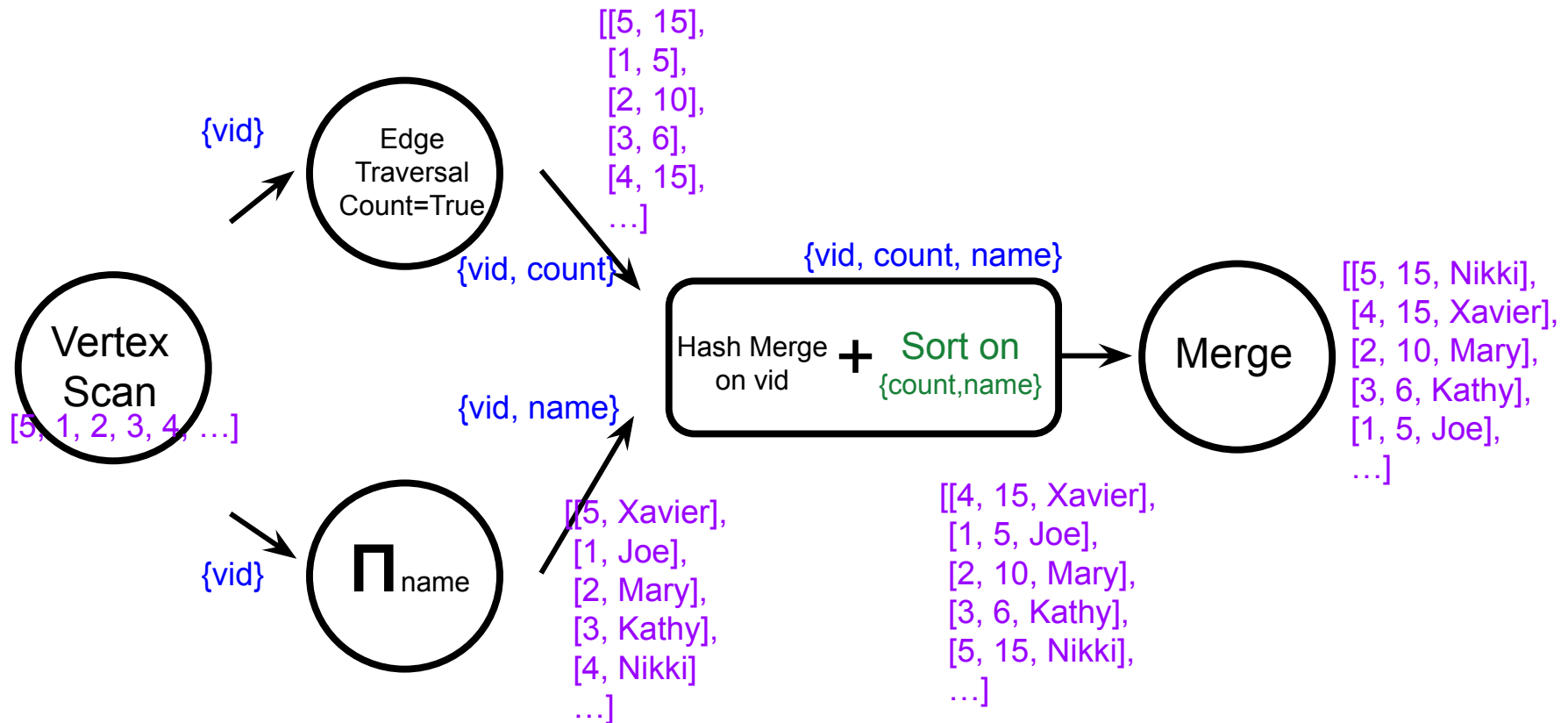


Parallelism in NEXUS

- Intra-operator parallelism:
 - Divide work across different instances of the same operator.
- **Inter-operator** parallelism:
 - Different operators process work concurrently.

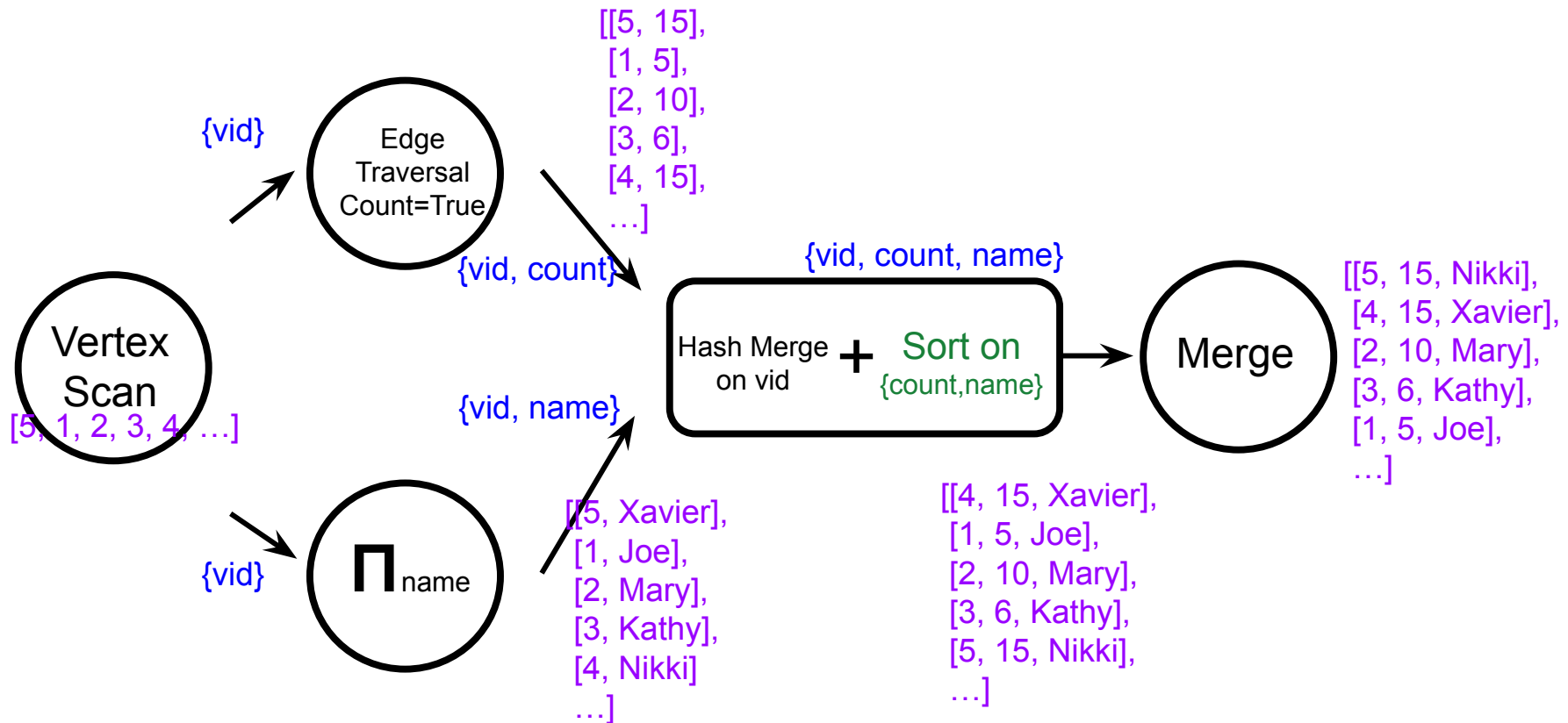
Pipelined Execution + Inter-Operator Parallelism

```
g.v().order.by(__.outE().count).by(name)
```



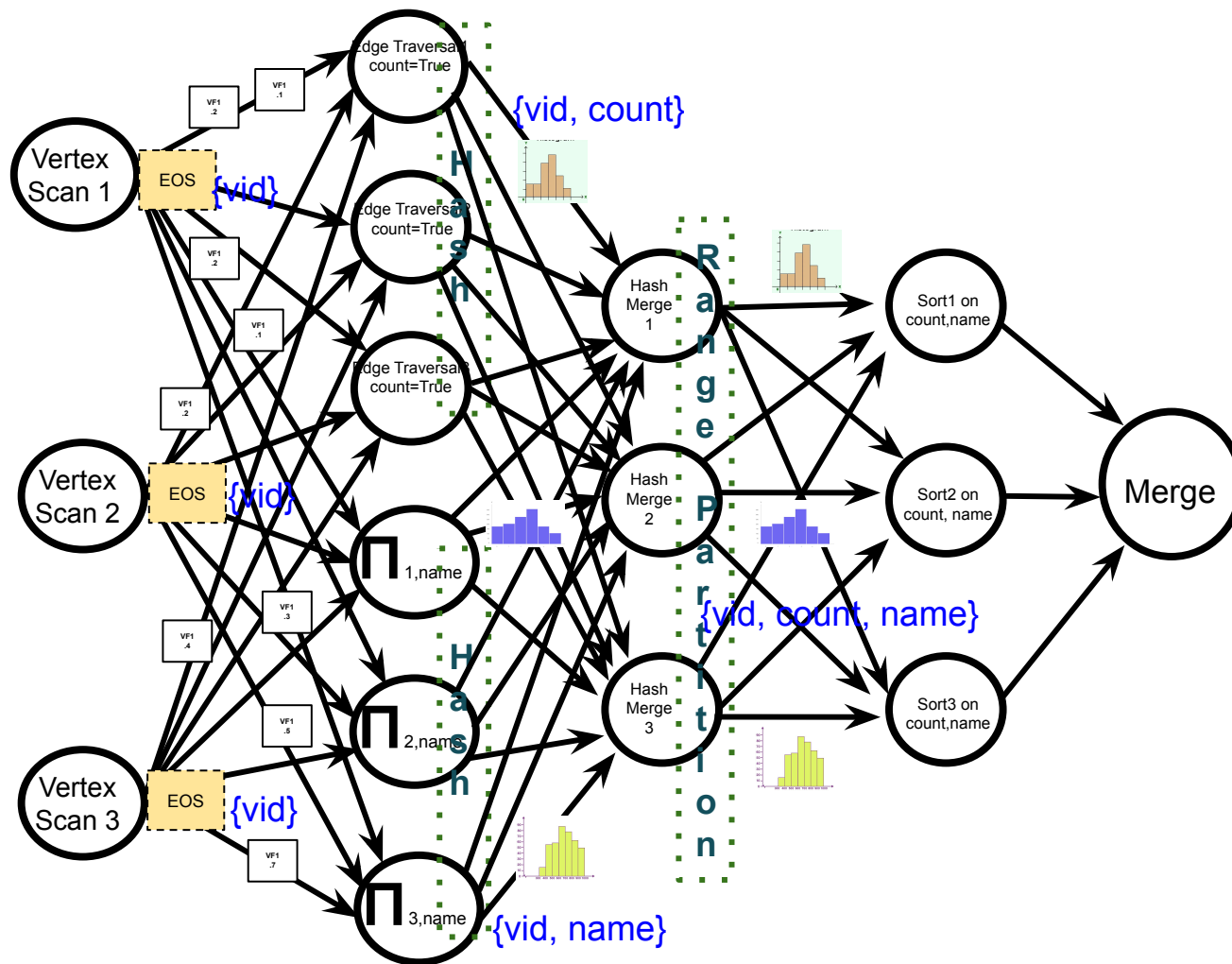
Inter-Operator Parallelism: How? Replicate data.

```
g.v().order.by(__.outE().count).by(name)
```



Pipelining
+
Intra-Operator
+
Inter-Operator



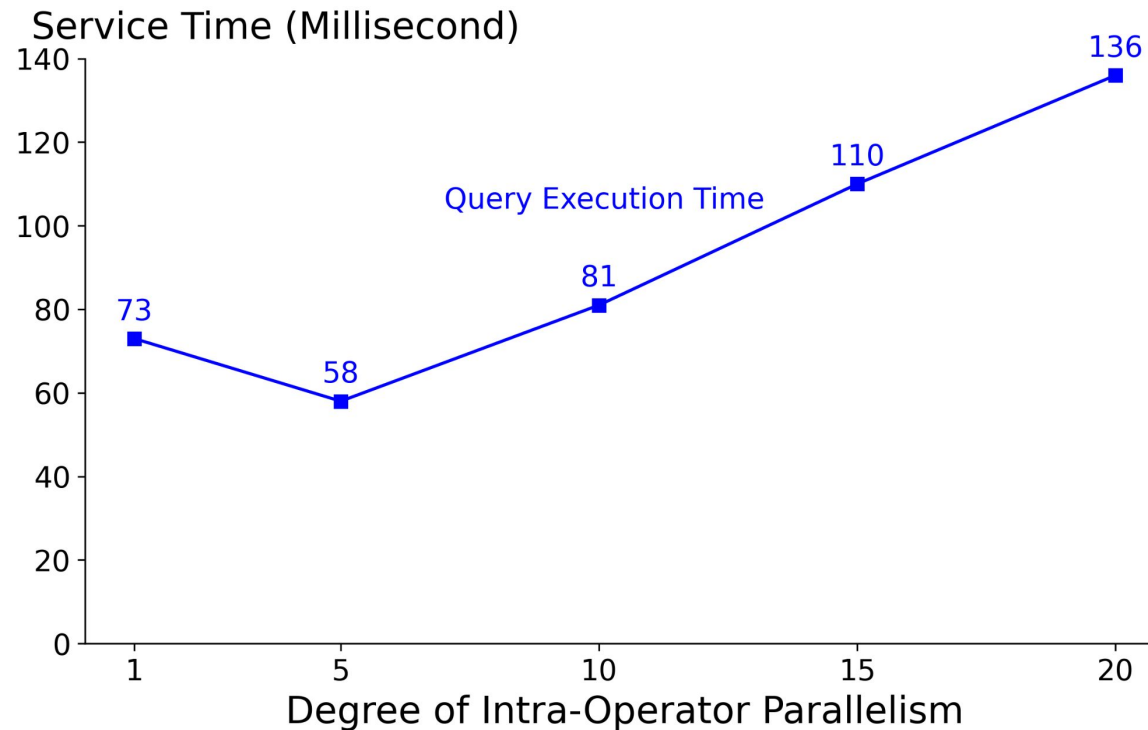


Software Modularity & Configurability: Split Library

- Configurable batch size by an operator instance.
- Discovery of operator instances as referenced by the query tree.
- Routing of network packets from a producing operator to a consuming operator.
 - Data for a query is in one or more network packets.
 - Split may be configured to sort the outgoing data.
- Control degree of intra-operator parallelism.
- Balancing the load across the instances of an operator.
- Query termination.
 - Notifies the xact coordinator of a query of the number of packets generated by the operator.
- Failure recovery.
 - Execution of a query completes even when one or more of its operators fail.
 - Split replays state of the failed operators only.

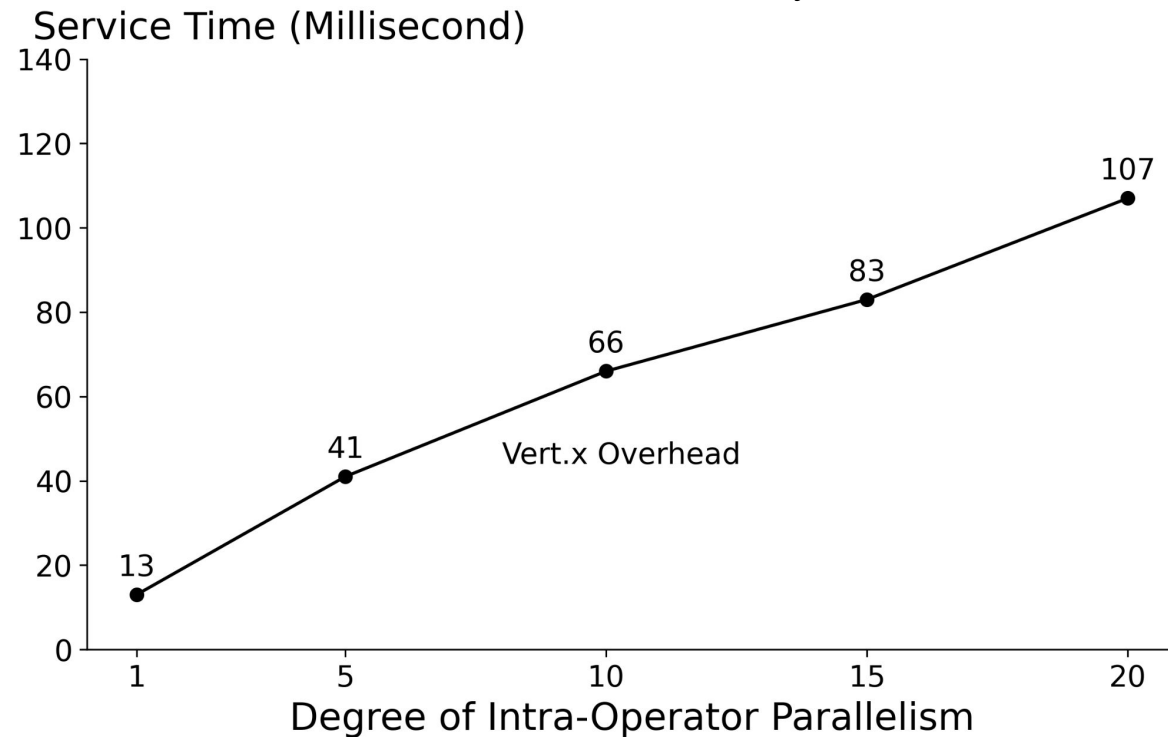
Query Execution Time

- A fixed environment:
 - Software: 6 VERTEX_FILTER, 20 EDGE_TRAVERSAL, 20 PROPERTY_MAP, 20 HASH_MERGE, 1 MERGE verticles. Airport DB: 3,634 Vertices, 52,639 Edges.
 - Hardware: CloudLab r650, 72 cores, 256 GB of memory.



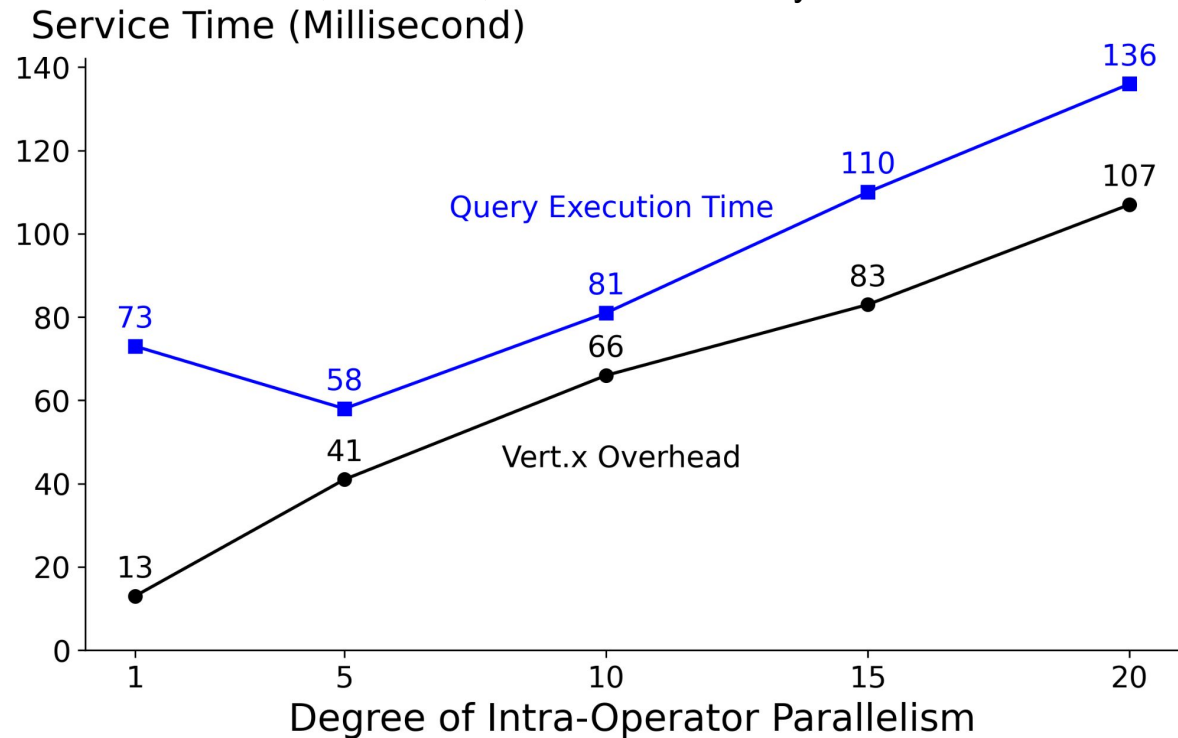
Vert.x Overhead

- A fixed environment:
 - Software: 6 VERTEX_FILTER, 20 EDGE_TRAVERSAL, 20 PROPERTY_MAP, 20 HASH_MERGE, 1 MERGE verticles. Batch size = 1000.
 - Hardware: CloudLab r650, 72 cores, 256 GB of memory.



Vert.x Overhead + Query Execution Time

- A fixed environment:
 - Software: 6 VERTEX_FILTER, 20 EDGE_TRAVERSAL, 20 PROPERTY_MAP, 20 HASH_MERGE, 1 MERGE verticles. Batch size = 1000.
 - Hardware: CloudLab r650, 72 cores, 256 GB of memory.





Shortest-Path

Graph RAG systems use the shortest-path to retrieve most relevant information from the knowledge graph. When a user asks a question, the system identifies relevant entities in the query and finds them in the graph. Next, it uses shortest-path to:

- Find connections: Direct paths between two or more entities in the knowledge graph.
- Contextual relevance: Edges might be weights by measure of relevant. Shortest path to find the most relevant not the most direct connections.
- Avoid over-traversal (efficiency): Shortest-path controls the retrieval process and prevents the system from traversing the entire graph aimlessly.

Why is shortest-path challenging? Cycles!

Gremlin implementation: `repeat()` operator, `simplePath()` for cycles.

Unweighted Shortest Path

- `g.V(src).repeat(out().simplePath()).until(hasId(dst)).path().limit(1)`
 - **src** and **dst** are vertex ids, e.g., `src=1`, `dst=25`.
- `T1.repeat(T2).until(P)` evaluates T2 until predicate P is true:
 - T1 produces vertices processed by `repeat`.
 - P may involve a traversal and computed in a pipelined manner.
- Repeat uses Split. For every input vertex, it constructs a network packet with a batch of vertices and T2. It dispatches the network packet for dataflow execution to the first operator of T2.
 - `#ConcurrentNavigations` controls how many T2 invocations are executing simultaneously.
 - Repeat maintains a queue of pending network packets.
 - For every execution of T2, Repeat evaluates the predicate P. With shortest path, it simply checks to see if the returned vertex equals **dst**.
 - If a vertex id satisfies the predicate, Repeat forwards the vertex id to the next operator, `limit(1)` in our example.
- Repeat may execute even though the execution of the query is complete.

Preventing Cycles: simplePath()

- The `simplePath()` step eliminates cycles. Today's implementations generate cycles, requiring `simplePath()` to detect and delete them.
- NEXUS avoids cycles. It combines `simplePath()` with a transform step such as `out()` or `in()` to avoid generating cycles. How?

Algorithm 1 OUT-SimplePath ()

Description: Combines `out()` with `simplepath()`.

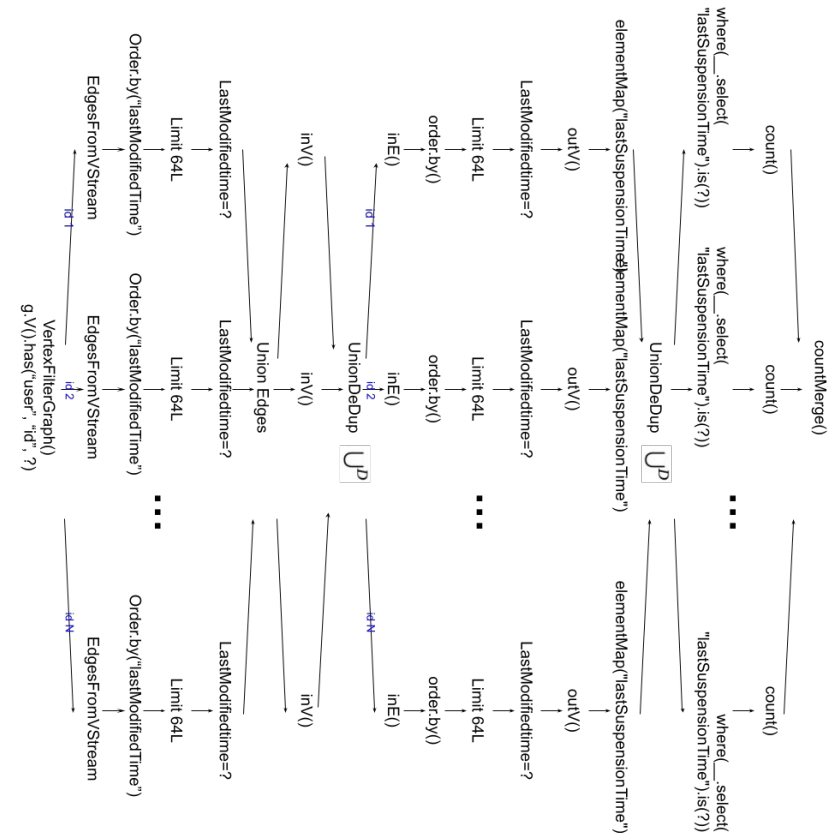
```
1:  $v_i \leftarrow$  Input vertex id
2:  $[P] \leftarrow$  Path that produced  $v_i$ 
3:  $H \leftarrow$  Construct a hash table on  $[P]$ , key is vertex id
4:  $S_p \leftarrow$  SplitInstance.initialize()
5:  $\{R\} \leftarrow v_i.out()$ 
6: for each  $v_r$  in  $\{R\}$  do
7:   Found = Probe  $H$  with  $v_r$ 
8:   if Found == false then
9:      $[P_{new}] = [P].append(v_r)$ 
10:     $elt = [v_r, [P_{new}]]$ 
11:     $S_p.send(elt)$ 
12:   end if
13: end for
14:  $S_p.close()$ 
```

Complex Queries: Inter/Intra-Operator Parallelism

- A query from eBay Risk application: Given a user, check whether the user is suspicious via IP address linking.

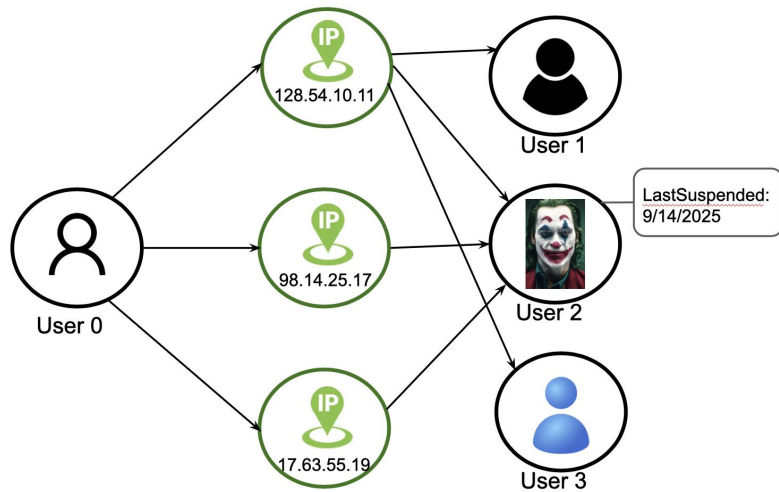
```
g.V().has("User", "id", ?) .
  local(__.outE("useSigninIpAttempt").order().by("
    lastModifiedTime", Order.desc).limit(64L).has(
      "lastModifiedTime", ?)).inV().dedup().
  local(__.inE("useSigninIpAttempt").order().by("
    lastModifiedTime", Order.desc).limit(64L).has(
      "lastModifiedTime", ?)).outV().elementMap("
    lastSuspensionTime").dedup().
  where(__.select("lastSuspensionTime").is(?)).
  count()
```

Execution Time: 30 msec

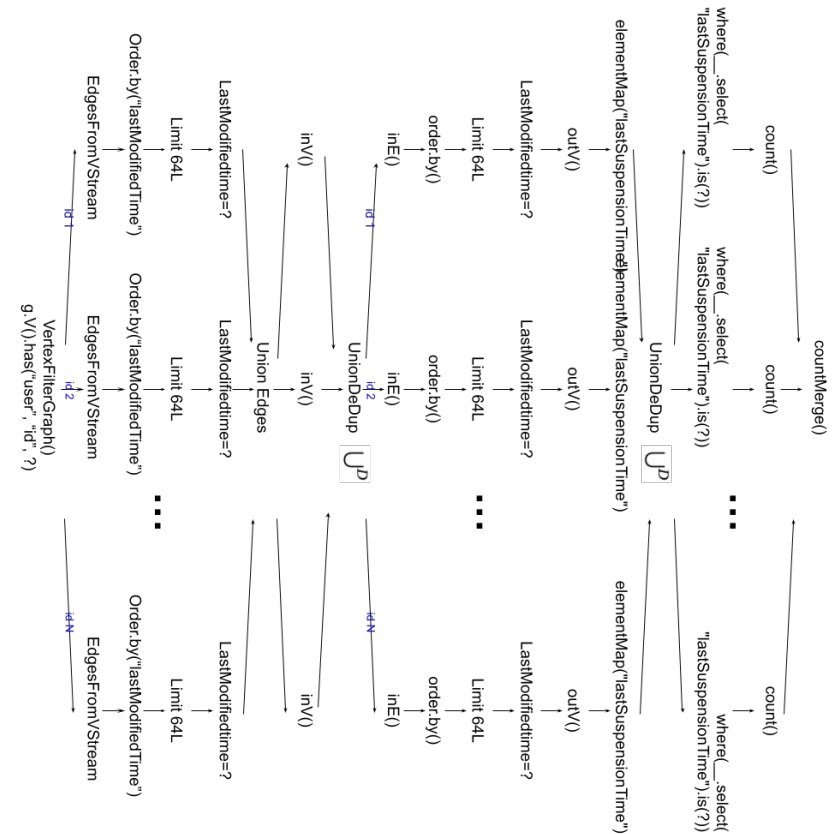


Complex Queries: Inter/Intra-Operator Parallelism

- A query from eBay Risk application: Given a user check whether the user is suspicious via IP address linking.

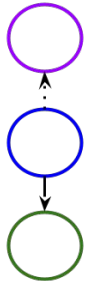


Execution Time: 30 msec



Match: eBay DataGov

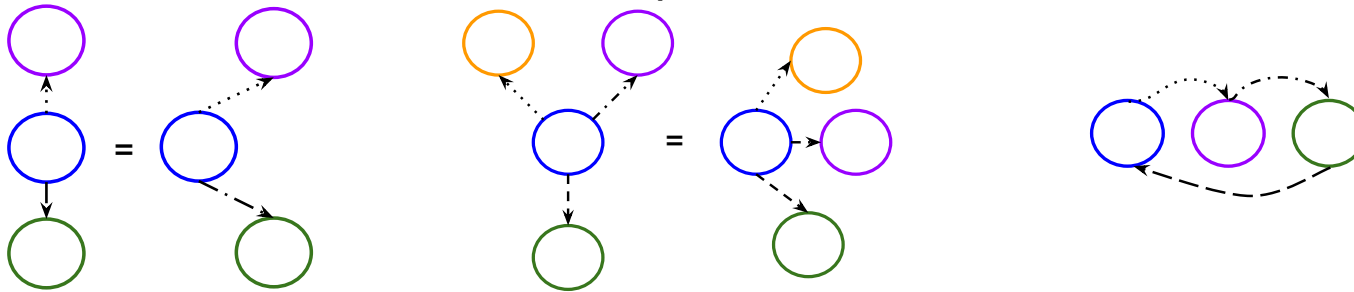
Fetch contract information ID and Infohub domain ID of all vendors who belong to an infohub domain with a contract.



```
g.V().match(
  __.as("VendorAPI").has("vtype","ebayontl:VendorAPI"),
  __.as("VendorAPI").out("ebayontl:belongsToInfohubDomainRelatedToVendorAPI").as("Infohub
Domain"),
  __.as("VendorAPI").out("ebayontl:hasContractInformationRelatedToVendorAPI").as("ContractI
nformation"),
  __.as("ContractInformation").values("ebayontl:ContractInformationId").as("ContractInformationI
d"),
  __.as("InfohubDomain").values("ebayontl:InfohubDomainId").as("InfohubDomainId")
).select("ContractInformationId","InfohubDomainId")
```

Declarative Pattern Matching

- Gremlin `match` defines a pattern of connected vertices and edges, requiring NEXUS to find all instances of that pattern.



- Example use cases:
 - Social Networking: Find all users who are friends with at least two people who live in the same city as the user
 - Fraud Detection:
 - A credit card (`cardA`) used to purchase from a store (`storeX`), which has the same owner (`ownerY`) as another card (`cardB`) that was recently reported as stolen.
 - A person (`person1`) who has a phone number (`phone1`) that is also associated with another person (`person2`) who lives at the same address.

Contributions

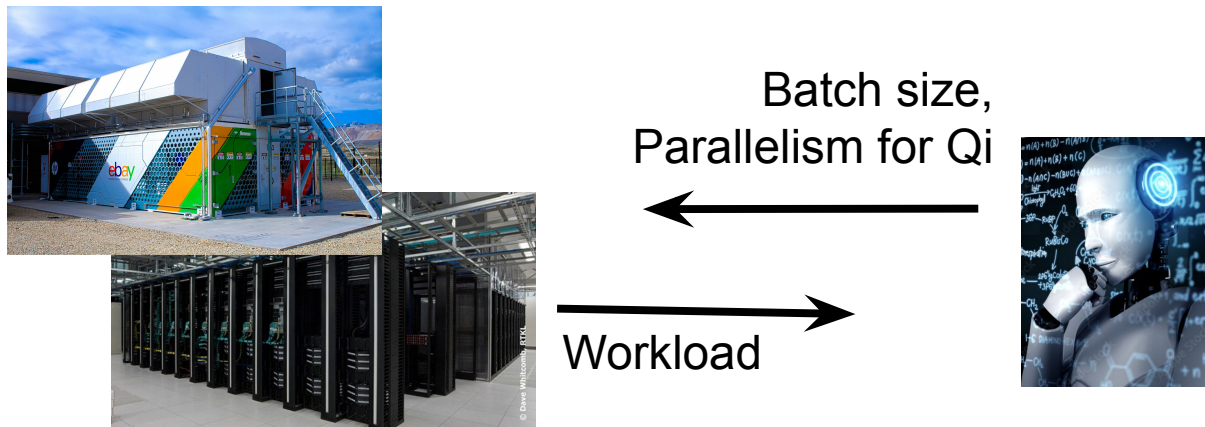
- We are the first to demonstrate a data flow execution paradigm for Gremlin queries:
 - Pipelining: Data flows across different operators of a query.
 - Batch size controls over-lapped processing by different operators of a query.
 - Inter-operator parallelism controls the number of operator instances working on behalf of a query concurrently.
 - Intra-operator parallelism allows multiple operators to work independently.
 - Shortest-path queries.
 - Declarative pattern matching.

Supported Gremlin Steps

- NEXUS uses the reference implementation of ThinkerPop to implement different Gremlin steps including:
 - Transform: `in()`, `out()`, `inE()`, `outE()`, `match()`, `values()`,
 - Filter: `has()`, `dedup()`,
 - Branch: `local()`,
 - Looping modulator: `repeat()`,
 - Path history modulator: `simplePath()`,
 - Map: `path()`, `elementMap()`, `valueMap()`,
 - Side-effect step: `order-by()`, `group-by()`, `count()`.
- The execution of a Gremlin query is made possible by internal NEXUS operators such as `Merge()`, `HashMerge()` and the Split library.

Future Research: Batch Size & Degree of Parallelism Tuner

- Use eBay workloads to dynamically configure the batch size and degree of parallelism for a query Q_i .



- Process sample queries from diverse eBay applications
 - Requires implementation of additional Gremlin steps.
 - Further demonstrates NEXUS's viability. Justifies an implementation of a development system.

Future Research

- Caches as an integral component of the dataflow execution paradigm:
 - 1-Hop query result caches.
 - Split to store/cache the result of an operator.
- Hybrid execution paradigm using dataflow for complex queries + traditional single-threaded JanusGraph for Simple queries.
- Failure handling: Extend Split library.
- Dataflow execution paradigm to use DiStash/FDB.
- Temporal graph database/knowledge graph.
- Graph RAG as an application of NEXUS.